《深入 OpenClaw》—— 目录与纲要

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **工具**: OpenClaw + OpenCode
> **生成日期**：2026-03-18 06:12
> **GitHub**：https://github.com/bunnyrui/OpenClaw-Book

前言

- 本书的写作动机与目标读者
- OpenClaw 项目概况：一个运行在自己设备上的个人 AI 助手
- 本书的组织结构与阅读建议
- 源码版本说明（基于 `v2026.3.9`）
- 预备知识：TypeScript 基础、Node.js 运行时概念、WebSocket 协议、基本的操作系统知识

第一部分：入门指南

> 从零开始，带你从安装到精通 OpenClaw。无需阅读源码，跟着做就能上手。

第 1 章 认识 OpenClaw

- **1.1 什么是 OpenClaw：个人 AI 助手的全新范式**
  - 1.1.1 OpenClaw 是什么，不是什么
  - 1.1.2 "本地优先"与"单用户"设计哲学
  - 1.1.3 与 ChatGPT / Claude.ai 等云端产品的区别
- **1.2 核心架构一览**
  - 1.2.1 一条消息的旅程：从你到 AI 再到你
  - 1.2.2 五大核心组件：Gateway、Agent、Channel、Tools、Skills
  - 1.2.3 架构图（文字版）
- **1.3 技术栈与项目生态**
  - 1.3.1 TypeScript + Node.js 22+：为什么选这个
  - 1.3.2 项目目录结构（鸟瞰）

- 1.3.3 OpenClaw 生态：ClawHub、Skills、Extensions、Native Apps

第 2 章 安装与快速上手

- **2.1 环境准备与安装**
  - 2.1.1 Node.js 22+ 与 pnpm 安装
  - 2.1.2 npm 全局安装 vs 从源码构建
  - 2.1.3 平台差异：macOS / Linux / Windows WSL
- **2.2 引导向导与首次配置**
  - 2.2.1 `openclaw onboard` 向导流程
  - 2.2.2 模型提供者配置
  - 2.2.3 Gateway 配置与首次通道连接
- **2.3 发送第一条消息**
  - 2.3.1 启动 Gateway
  - 2.3.2 通过 CLI 发送消息
  - 2.3.3 幕后发生了什么（简化版数据流）

第 3 章 通道连接实战

- **[3.1 Telegram Bot 完整配置](ZH/第03章-通道连接实战/3.1-Telegram Bot完整配置.md)**
  - 3.1.1 BotFather 创建 Bot
  - 3.1.2 openclaw.json 配置与 Webhook
  - 3.1.3 群组权限与测试
- **3.2 Discord 与 Slack 接入**
  - 3.2.1 Discord Bot 完整配置
  - 3.2.2 Slack Bolt 集成
  - 3.2.3 WhatsApp / Signal / iMessage 简介
- **3.3 多通道策略与群组管理**
  - 3.3.1 同时运行多个通道
  - 3.3.2 群组策略：Mention Gating / Command Gating
  - 3.3.3 Allowlist 与通道专属配置

第 4 章 对话与工具使用

- **4.1 对话技巧与命令**
  - 4.1.1 聊天命令：/new /reset /compact /think /model
  - 4.1.2 有效对话技巧
  - 4.1.3 会话管理（用户视角）

- **4.2 Bash 工具实战**
  - 4.2.1 文件管理与代码执行
  - 4.2.2 系统管理与进程控制
  - 4.2.3 安全模型：什么能做什么不能做
- **4.3 浏览器工具与 Canvas 画布**
  - 4.3.1 浏览器工具：导航、截图、数据提取
  - 4.3.2 Canvas 画布：A2UI 概念
  - 4.3.3 实战示例

第 5 章 定时任务与自动化

- **5.1 Cron 定时任务入门**
  - 5.1.1 Cron 表达式语法详解
  - 5.1.2 创建与管理定时任务
  - 5.1.3 投递计划：结果发送到哪里
- **5.2 Webhook 与事件驱动**
  - 5.2.1 Webhook 触发器
  - 5.2.2 Gmail Pub/Sub 邮件钩子
  - 5.2.3 事件驱动自动化模式
- **5.3 自动化实战案例**
  - 5.3.1 天气预报 / 服务器监控
  - 5.3.2 日报生成 / 新闻摘要
  - 5.3.3 代码备份 / 自动部署

第 6 章 Skills 生态

- **6.1 Skills 概念与 ClawHub**
  - 6.1.1 什么是 Skill：可热插拔的 Agent 能力模块
  - 6.1.2 Skill 类型：Bundled / Managed / Workspace
  - 6.1.3 ClawHub 市场
- **6.2 实战：常用 Skills 详解**
  - 6.2.1 coding-agent / github / weather
  - 6.2.2 browser / discord / canvas
  - 6.2.3 安装、配置与使用示例
- **6.3 编写自定义 Skill**
  - 6.3.1 SKILL.md 文件规范
  - 6.3.2 从零编写一个自定义 Skill
  - 6.3.3 测试、调试与发布

第 7 章 模型配置与优化

- **7.1 多提供者配置与 API 密钥**
  - 7.1.1 Anthropic / OpenAI / Google / 本地模型配置
  - 7.1.2 API 密钥管理与 Auth Profile
  - 7.1.3 OAuth 认证流程
- **7.2 回退链与性价比优化**
  - 7.2.1 三级回退策略
  - 7.2.2 模型扫描与免费模型发现
  - 7.2.3 成本优化技巧
- **7.3 中国 AI 模型接入**
  - 7.3.1 DeepSeek / 通义千问 / 月之暗面
  - 7.3.2 智谱 / 豆包 配置
  - 7.3.3 GFW 环境下的使用策略

第 8 章 中国平台与部署

- **8.1 飞书 / 钉钉 / 企业微信集成**
  - 8.1.1 飞书 / Lark 扩展配置
  - 8.1.2 钉钉扩展配置
  - 8.1.3 企业微信扩展配置
- **8.2 国内云部署实战**
  - 8.2.1 阿里云 / 腾讯云部署
  - 8.2.2 安全组 / 域名 / 反向代理
  - 8.2.3 ICP 备案与成本估算
- **8.3 Docker 部署与生产配置**
  - 8.3.1 Docker Compose 部署
  - 8.3.2 环境变量与持久化
  - 8.3.3 生产环境安全清单

第 9 章 多 Agent 实战

- **9.1 多 Agent 配置与路由**
  - 9.1.1 为什么需要多个 Agent
  - 9.1.2 agents.list[] 配置与路由规则
  - 9.1.3 Agent 专属模型 / Skill / 工具设置
- **9.2 子代理与跨 Agent 通信**
  - 9.2.1 Sub-agent 生成与通信

    - 9.2.2 跨 Agent 消息传递
    - 9.2.3 Agent Scope 与隔离
- **9.3 团队协作与实战案例**
    - 9.3.1 代码审查 + 部署 Agent
    - 9.3.2 客服 + 升级 Agent
    - 9.3.3 研究 + 写作 Agent

第 10 章 高级技巧与最佳实践

- **10.1 记忆系统与知识管理**
    - 10.1.1 长期记忆 MEMORY.md
    - 10.1.2 向量搜索与记忆管理
    - 10.1.3 Embedding 提供者配置
- **10.2 安全加固与权限管理**
    - 10.2.1 DM 配对与沙箱模式
    - 10.2.2 工具策略：Allow/Deny 列表
    - 10.2.3 SOUL 安全与生产加固
- **10.3 定制与常见问题**
    - 10.3.1 AGENTS.md / SOUL.md 定制
    - 10.3.2 常见问题排查 FAQ
    - 10.3.3 接下来读什么

第二部分：Gateway 控制平面

第 11 章 Gateway 服务器架构

- **11.1 Gateway 的角色与设计目标**
    - 11.1.1 "控制平面"（Control Plane）的含义
    - 11.1.2 单 Gateway 实例约束
    - 11.1.3 Gateway 的职责边界
- **11.2 服务器启动流程源码分析**
    - 11.2.1 入口文件 `src/entry.ts`
    - 11.2.2 Gateway CLI 启动
    - 11.2.3 服务器启动序列
    - 11.2.4 服务器实现类
- **11.3 WebSocket 服务器实现**

    - 11.3.1 WebSocket 传输层
    - 11.3.2 WebSocket 运行时管理
    - 11.3.3 连接生命周期
    - 11.3.4 帧协议格式
- **11.4 HTTP 层**
    - 11.4.1 HTTP 服务器搭建
    - 11.4.2 OpenAI 兼容 HTTP 接口
    - 11.4.3 Open Responses API
    - 11.4.4 工具调用 HTTP 接口

第 12 章 Gateway 协议与类型系统

- **12.1 协议设计哲学**
    - 12.1.1 为什么选择 WebSocket 而非 REST
    - 12.1.2 请求-响应 + 服务器推送事件混合模式
    - 12.1.3 幂等键机制
- **12.2 TypeBox 类型模式**
    - 12.2.1 协议 Schema 定义
    - 12.2.2 从 TypeBox 生成 JSON Schema
    - 12.2.3 从 JSON Schema 生成 Swift 模型
    - 12.2.4 协议一致性校验
- **12.3 核心方法与事件**
    - 12.3.1 请求方法一览
    - 12.3.2 `connect` / `agent` / `send` / `sessions.*`
    - 12.3.3 事件类型
- **12.4 认证与授权**
    - 12.4.1 Gateway Token 认证
    - 12.4.2 设备配对机制
    - 12.4.3 Origin 检查

第 13 章 会话管理

- **13.1 会话模型设计**
- **13.2 会话路由源码分析**
- **13.3 会话生命周期**
- **13.4 会话裁剪**
- **13.5 会话间通信**

第 14 章 通道路由与消息分发

- 14.1 通道注册表
- 14.2 入站消息处理流水线
- 14.3 出站消息处理
- 14.4 多 Agent 路由

第三部分：AI Agent 运行时

第 15 章 Pi Agent 运行时核心

- 15.1 Pi Agent 是什么
- 15.2 Agent Loop 端到端解析
- 15.3 队列与并发控制
- 15.4 超时与中止机制

第 16 章 模型提供者与故障转移

- 16.1 模型选择机制
- 16.2 Auth Profile 与凭据管理
- 16.3 模型目录与配置
- 16.4 故障转移错误处理

第 17 章 系统提示词与上下文组装

- 17.1 系统提示词的构建
- 17.2 工作区与上下文文件注入
- 17.3 身份系统
- 17.4 上下文压缩
- [17.5 Context Engine 插件系统](ZH/第17章-系统提示词与上下文组装/17.5-Context Engine插件系统.md)

第 18 章 流式传输与分块回复

- 18.1 流式传输架构
- 18.2 块流

- 18.3 合并与人性化节奏
- 18.4 Telegram 草稿流
- 18.5 回复塑形与抑制

第四部分：多通道消息系统

第 19 章 通道适配器抽象层

- 19.1 通道适配器设计模式
- 19.2 入站消息规范化
- 19.3 出站消息适配

第 20 章 核心通道实现深入

- 20.1 WhatsApp 通道
- 20.2 Telegram 通道
- 20.3 Discord 通道
- 20.4 Slack 通道
- 20.5 其他核心通道

第 21 章 通道扩展机制

- 21.1 扩展架构设计
- 21.2 扩展 API 表面
- 21.3 代表性扩展实现解析
- 21.4 开发自定义扩展

第五部分：工具系统与自动化

第 22 章 工具系统总览

- 22.1 工具系统架构
- 22.2 工具执行运行时
- 22.3 工具权限与策略

- 22.4 工具与 Agent 的协作

第 23 章 Bash 工具与进程管理

- 23.1 Bash 执行引擎
- 23.2 进程管理
- 23.3 Shell 工具与共享基础设施

第 24 章 浏览器控制

- 24.1 浏览器架构概述
- 24.2 CDP 层实现
- 24.3 Playwright 层实现
- 24.4 浏览器服务器

第 25 章 Canvas 与 A2UI

- 25.1 Canvas 概念
- 25.2 Canvas Host 实现
- 25.3 Canvas 工具

第 26 章 Cron 调度与自动化

- 26.1 Cron 系统设计
- 26.2 Cron 作业执行
- 26.3 Webhook 与 Gmail Pub/Sub

第 27 章 节点系统

- 27.1 节点概念
- 27.2 节点注册与发现
- 27.3 节点 Host 实现

第六部分：记忆、技能与生态系统

第 28 章 记忆系统

- 28.1 记忆模型设计
- 28.2 向量记忆搜索
- 28.3 混合搜索
- 28.4 高级记忆功能

第 29 章 技能系统

- 29.1 技能平台设计
- 29.2 技能结构
- 29.3 技能安装与管理
- 29.4 ClawHub 技能注册中心

第 30 章 钩子系统

- 30.1 内部钩子
- 30.2 插件钩子

第七部分：安全、配置与基础设施

第 31 章 配置系统

- 31.1 配置加载与解析
- 31.2 配置类型系统详解
- 31.3 配置热重载
- 31.4 遗留配置迁移
- 31.5 环境变量

第 32 章 安全模型

- 32.1 安全设计原则
- 32.2 DM 配对系统
- 32.3 沙箱机制

- 32.4 安全审计
- 32.5 SOUL 安全

第 33 章 CLI 命令行工具

- 33.1 CLI 架构
- 33.2 核心命令解析
- 33.3 聊天命令

第 34 章 基础设施

- 34.1 日志系统
- 34.2 媒体管道
- 34.3 链接理解与媒体理解
- 34.4 TTS 文本转语音
- 34.5 投票系统

第八部分：客户端应用与 Web UI

第 35 章 Web 控制台 UI

- 35.1 UI 技术选型
- 35.2 控制台 UI 架构
- 35.3 WebChat 实现

第 36 章 原生客户端应用

- 36.1 macOS 应用
- 36.2 iOS 节点应用
- 36.3 Android 节点应用
- 36.4 共享组件

第九部分：部署与运维

第 37 章 部署方案

- 37.1 本地部署
- 37.2 Docker 部署
- 37.3 远程访问
- 37.4 Nix 声明式部署
- 37.5 VPS 部署

第 38 章 监控与故障排查

- 38.1 健康检查
- 38.2 在线追踪
- 38.3 用量追踪
- 38.4 故障排查

第十部分：高级主题与实践

第 39 章 多代理架构

- 39.1 多代理设计
- 39.2 子代理
- 39.3 多代理沙箱工具

第 40 章 ACP（Agent Communication Protocol）

- 40.1 ACP 协议概述
- 40.2 ACP 会话管理与运行时
- 40.3 ACP 控制面与集成模式

第 41 章 TUI（终端用户界面）

- 41.1 TUI 架构
- 41.2 TUI 交互机制
- 41.3 TUI 高级特性

第 42 章 实战项目：构建你自己的 AI 助手

- 42.1 项目规划
- 42.2 核心功能实现
- 42.3 通道集成
- 42.4 高级功能
- 42.5 部署上线

附录

附录 A：OpenClaw 配置参考

- 完整配置 JSON5 示例
- 所有配置键速查表

附录 B：Gateway WebSocket 协议参考

- 所有请求方法与参数
- 所有事件类型与负载

附录 C：工具定义参考

- 所有内置工具的 Schema 与用途

附录 D：源码导航图

- 关键模块依赖关系图
- 数据流序列图

附录 E：术语表

- 本书出现的全部专业术语中英文对照与解释

关于估算

部分	章节数	预估字数
第一部分：入门指南	10 章	~80,000 字
第二部分：Gateway 控制平面	4 章	~16,000 字
第三部分：AI Agent 运行时	4 章	~16,000 字
第四部分：多通道消息系统	3 章	~12,000 字
第五部分：工具系统与自动化	6 章	~18,000 字
第六部分：记忆、技能与生态	3 章	~10,000 字
第七部分：安全、配置与基础设施	4 章	~12,000 字
第八部分：客户端应用与 Web UI	2 章	~6,000 字
第九部分：部署与运维	2 章	~6,000 字
第十部分：高级主题与实践	4 章	~12,000 字
附录	5 项	~4,000 字
合计	**42 章 + 5 附录**	**~192,000 字**